UAA7 – Bases de données

SQL DDL

Exercices supplémentaires

Ces exercices complètent la section 5.1 du cours (Data Definition Language). Ils travaillent :

- le choix des types de données (INT, SMALLINT, DECIMAL(p,q), VARCHAR, CHAR, DATE, TIME, TIMESTAMP…) ;
- la création de schémas et de tables (CREATE SCHEMA, CREATE TABLE) ;
- les contraintes : PRIMARY KEY (simple et multi-colonnes), UNIQUE, NOT NULL, FOREIGN KEY, CHECK;
- les comportements à la suppression : ON DELETE RESTRICT, CASCADE et SET NULL ;
- la suppression d'une table (DROP TABLE) ;
- la modification d'une table existante (ALTER TABLE) ;
- la traduction d'un schéma conceptuel en tables (chapitre 4).

Corrigé

Toutes les requêtes du corrigé ont été testées sur PostgreSQL.

Exercice 1

	Type	Justification
a	VARCHAR(20)	Contient un 0 initial, des espaces, parfois un « + ». On ne calcule jamais avec un numéro.
b	INT (ou SMALLINT)	Quatre chiffres, sans zéro initial en Belgique. On ajoute CHECK (codePostal BETWEEN 1000 AND 9999).
c	DECIMAL(6,2)	6 chiffres au total, dont 2 après la virgule : maximum 9 999,99.
d	DECIMAL(3,1)	Maximum 20,0 : 3 chiffres dont 1 après la virgule. On ajoute CHECK (moyenne BETWEEN 0 AND 20).
e	CHAR(11)	Longueur toujours identique, et un nombre perdrait le 0 initial.
f	DATE	Année, mois et jour.
g	TIME	Heure, minute et seconde, sans date.
h	TIMESTAMP	Date et heure ensemble.
i	SMALLINT	Petit entier : une classe ne dépasse jamais 32 767 élèves.
j	BIT	Deux valeurs possibles : 0 ou 1 (BOOLEAN sur PostgreSQL).
k	INTERVAL	Durée entre deux dates, en années, mois ou jours.
l	INT	Dépasse la capacité d'un SMALLINT (32 767).

a) Maximum 999,99 et minimum −999,99 : 5 chiffres, dont 2 après la virgule, donc 3 avant.

b) DECIMAL(4,1) autorise au maximum 999,9. **125,75** a deux décimales : le SGBD l'arrondit à 125,8, la donnée est donc modifiée sans avertissement. **1 250** est refusé : il faudrait 4 chiffres avant la virgule. **99,9** est accepté.

c) Le 0 initial disparaîtrait (0471… deviendrait 471…), les espaces et le « + » seraient impossibles, et un numéro n'est pas une quantité sur laquelle on calcule.

Exercice 2

```
-- a
quantite INT NOT NULL CHECK (quantite > 0)
-- b
CONSTRAINT chk_remise CHECK (remise BETWEEN 0 AND 50)
-- c
CONSTRAINT chk_genre CHECK (genre IN ('Action', 'Comédie', 'Drame', 'Animation'))
-- d
CONSTRAINT uk_email UNIQUE (email)
-- e
PRIMARY KEY (eleve, cours)
-- f  (contrainte portant sur deux colonnes)
CONSTRAINT chk_periode CHECK (dateFin >= dateDebut)
-- g  (_ remplace exactement un caractère)
CONSTRAINT chk_plaque CHECK (plaque LIKE '_-___-___')
-- h
nom VARCHAR(40) NOT NULL
-- i  (identifiant secondaire sur deux colonnes)
CONSTRAINT uk_local UNIQUE (etage, nom)
-- j
salaire DECIMAL(7,2)
```

Pour la g, LIKE '_-___-___' vérifie la forme mais pas la nature des caractères : « X-123-ABC » serait aussi accepté. C'est suffisant à ce niveau.

Exercice 3

	Erreur et explication
1	INSCRIPTION est créée **avant** ELEVE et COURS : ses clés étrangères visent des tables qui n'existent pas encore. Il faut la créer en dernier.
2	PRIMARY KEY (eleve) : un élève ne pourrait s'inscrire qu'à un seul cours. La clé primaire est (eleve, cours).
3	Types incompatibles : INSCRIPTION.eleve est un INT, ELEVE.matricule un VARCHAR(8). Une clé étrangère et la clé qu'elle vise doivent avoir le même type.
4	Virgule manquante après nom VARCHAR(40) NOT NULL.
5	DECIMAL(2,1) ne dépasse pas 9,9 : une moyenne sur 20 est impossible. Il faut DECIMAL(3,1).
6	classe est NOT NULL avec ON DELETE SET NULL : c'est contradictoire. Il faut rendre la colonne facultative, ou choisir ON DELETE RESTRICT.
7	La clé primaire de SALLE compte deux colonnes : la clé étrangère doit reprendre (salleEtage, salleNumero), pas seulement salleNumero.
8	Le CHECK porte sur nbHeures, qui n'existe pas dans COURS : il faut déclarer cette colonne.

Script corrigé :

```
CREATE TABLE ELEVE (
    matricule INT PRIMARY KEY,
    nom       VARCHAR(40) NOT NULL,
    prenom    VARCHAR(40) NOT NULL,
    moyenne   DECIMAL(3,1),
    classe    VARCHAR(5) REFERENCES CLASSE (code) ON DELETE SET NULL,
    CONSTRAINT chk_moyenne CHECK (moyenne BETWEEN 0 AND 20)
);
CREATE TABLE COURS (
    code       VARCHAR(10) PRIMARY KEY,
    intitule   VARCHAR(50) NOT NULL,
    nbHeures   SMALLINT,
    salleEtage  SMALLINT,
    salleNumero SMALLINT,
    CONSTRAINT fk_cours_salle FOREIGN KEY (salleEtage, salleNumero) REFERENCES SALLE (etage,
numero),
    CONSTRAINT chk_heures CHECK (nbHeures > 0)
);
CREATE TABLE INSCRIPTION (
    eleve         INT NOT NULL,
    cours         VARCHAR(10) NOT NULL,
    dateInscription DATE NOT NULL,
    PRIMARY KEY (eleve, cours),
    FOREIGN KEY (eleve) REFERENCES ELEVE (matricule),
    FOREIGN KEY (cours) REFERENCES COURS (code)
);
```

Exercice 4

```
CREATE SCHEMA BIBLIO;
SET search_path TO BIBLIO;

CREATE TABLE AUTEUR (
   idAuteur    INT PRIMARY KEY,
   nom         VARCHAR(40) NOT NULL,
   prenom      VARCHAR(40) NOT NULL,
   nationalite VARCHAR(30)
);

CREATE TABLE LIVRE (
   isbn         CHAR(13) PRIMARY KEY,
   titre        VARCHAR(100) NOT NULL,
   anneeParution INT,
   nbPages      INT,
   genre        VARCHAR(20),
   CONSTRAINT chk_annee  CHECK (anneeParution BETWEEN 1450 AND 2100),
   CONSTRAINT chk_pages  CHECK (nbPages > 0),
   CONSTRAINT chk_genre  CHECK (genre IN ('Roman', 'BD', 'Manga', 'Documentaire', 'Poésie'))
);

CREATE TABLE ECRIT (
   livre  CHAR(13) NOT NULL,
   auteur INT NOT NULL,
   PRIMARY KEY (livre, auteur),
   CONSTRAINT fk_ecrit_livre  FOREIGN KEY (livre)  REFERENCES LIVRE (isbn) ON DELETE CASCADE,
   CONSTRAINT fk_ecrit_auteur FOREIGN KEY (auteur) REFERENCES AUTEUR (idAuteur) ON DELETE
RESTRICT
);

CREATE TABLE EXEMPLAIRE (
   livre         CHAR(13) NOT NULL,
   numExemplaire  SMALLINT NOT NULL,
   etat          VARCHAR(10) NOT NULL,
   dateAchat     DATE NOT NULL,
   PRIMARY KEY (livre, numExemplaire),
   CONSTRAINT fk_exemplaire_livre FOREIGN KEY (livre) REFERENCES LIVRE (isbn),
   CONSTRAINT chk_etat CHECK (etat IN ('Neuf', 'Bon', 'Abîmé'))
);

CREATE TABLE MEMBRE (
   numCarte VARCHAR(9) PRIMARY KEY,
   nom      VARCHAR(40) NOT NULL,
   prenom   VARCHAR(40) NOT NULL,
   email    VARCHAR(60) NOT NULL,
   classe   VARCHAR(5),
   CONSTRAINT uk_email UNIQUE (email),
   CONSTRAINT chk_numCarte CHECK (numCarte LIKE 'BIB-____')
);

CREATE TABLE EMPRUNT (
   livre         CHAR(13) NOT NULL,
```

```
    numExemplaire  SMALLINT NOT NULL,
    dateEmprunt    DATE NOT NULL,
    dateRetour     DATE,
    membre         VARCHAR(9),
    PRIMARY KEY (livre, numExemplaire, dateEmprunt),
    CONSTRAINT fk_emprunt_exemplaire FOREIGN KEY (livre, numExemplaire)
       REFERENCES EXEMPLAIRE (livre, numExemplaire),
    CONSTRAINT fk_emprunt_membre FOREIGN KEY (membre)
       REFERENCES MEMBRE (numCarte) ON DELETE SET NULL,
    CONSTRAINT chk_dates CHECK (dateRetour >= dateEmprunt)
);
```

e) L'identifiant d'EMPRUNT est (livre, numExemplaire, dateEmprunt) : un exemplaire est identifié par deux colonnes et ne peut être emprunté qu'une fois par jour. La clé étrangère vers EXEMPLAIRE compte donc **deux colonnes**. La colonne membre est **facultative**, puisque ON DELETE SET NULL doit pouvoir la vider pour conserver l'historique.

f) Une table doit exister avant d'être référencée. On crée d'abord AUTEUR et LIVRE, puis ECRIT et EXEMPLAIRE (qui visent LIVRE), puis MEMBRE, et enfin EMPRUNT (qui vise EXEMPLAIRE et MEMBRE). MEMBRE peut aussi être créée en premier.

Exercice 5

a) Schéma logique :

- MONITEUR (matricule, nom, prenom, dateEngagement)
- ELEVE (numEleve, nom, prenom, dateNais, telephone, #moniteur)
- VEHICULE (plaque, marque, modele, boite)
- HABILITATION (#moniteur, #vehicule, dateHabilitation)
- LECON (numLecon, debut, duree, #eleve, #moniteur, #vehicule)

Clés primaires : matricule, numEleve, plaque, (moniteur, vehicule) et numLecon.

b) L'habilitation est une association plusieurs-à-plusieurs porteuse de données : elle devient la table HABILITATION. Sa clé primaire est la concaténation des deux clés étrangères, (moniteur, vehicule). Le suivi par un moniteur référent est un un-à-plusieurs : il devient simplement la clé étrangère moniteur dans ELEVE.

c) Script :

```
CREATE TABLE MONITEUR (
   matricule      VARCHAR(10) PRIMARY KEY,
   nom            VARCHAR(40) NOT NULL,
   prenom         VARCHAR(40) NOT NULL,
   dateEngagement DATE NOT NULL
);

CREATE TABLE ELEVE (
   numEleve   INT PRIMARY KEY,
   nom        VARCHAR(40) NOT NULL,
   prenom     VARCHAR(40) NOT NULL,
   dateNais   DATE NOT NULL,
   telephone  VARCHAR(20),
   moniteur   VARCHAR(10) NOT NULL,
   CONSTRAINT fk_eleve_moniteur FOREIGN KEY (moniteur) REFERENCES MONITEUR (matricule)
);

CREATE TABLE VEHICULE (
   plaque     VARCHAR(9) PRIMARY KEY,
   marque     VARCHAR(30) NOT NULL,
   modele     VARCHAR(30) NOT NULL,
   boite      VARCHAR(11) NOT NULL,
   CONSTRAINT chk_plaque CHECK (plaque LIKE '_-___-___'),
   CONSTRAINT chk_boite  CHECK (boite IN ('Manuelle', 'Automatique'))
);

CREATE TABLE HABILITATION (
   moniteur        VARCHAR(10) NOT NULL,
   vehicule        VARCHAR(9) NOT NULL,
   dateHabilitation DATE NOT NULL,
   PRIMARY KEY (moniteur, vehicule),
   CONSTRAINT fk_hab_moniteur FOREIGN KEY (moniteur) REFERENCES MONITEUR (matricule) ON
DELETE CASCADE,
   CONSTRAINT fk_hab_vehicule FOREIGN KEY (vehicule) REFERENCES VEHICULE (plaque) ON DELETE
CASCADE
);
```

```
CREATE TABLE LECON (
   numLecon  INT PRIMARY KEY,
   debut     TIMESTAMP NOT NULL,
   duree     SMALLINT NOT NULL,
   eleve     INT NOT NULL,
   moniteur  VARCHAR(10) NOT NULL,
   vehicule  VARCHAR(9) NOT NULL,
   CONSTRAINT chk_duree CHECK (duree IN (60, 90, 120)),
   CONSTRAINT fk_lecon_eleve    FOREIGN KEY (eleve)    REFERENCES ELEVE (numEleve),
   CONSTRAINT fk_lecon_moniteur FOREIGN KEY (moniteur) REFERENCES MONITEUR (matricule),
   CONSTRAINT fk_lecon_vehicule FOREIGN KEY (vehicule) REFERENCES VEHICULE (plaque)
);
```

Exercice 6

	Comportement	Justification
a	CASCADE	Une ligne de détail n'a aucun sens sans sa commande.
b	RESTRICT	Supprimer le client supprimerait ou rendrait orphelines des factures qu'on doit conserver.
c	SET NULL	Le ticket reste dans l'historique mais n'est plus rattaché à personne.
d	CASCADE	Les photos appartiennent à l'album.
e	RESTRICT	On refuse la suppression tant que des produits utilisent la catégorie.
f	SET NULL	La classe existe toujours, sans titulaire pour l'instant.
g	CASCADE	Les scores d'un joueur disparu n'ont plus d'intérêt (même choix que la table SCORE du cours).

Question : non. SET NULL doit pouvoir écrire NULL dans technicien : la colonne doit être facultative. Si elle est NOT NULL, la suppression du technicien échoue.

Exercice 7

a)

```
ALTER TABLE SEANCE ADD FOREIGN KEY (film)  REFERENCES FILM (idFilm);
ALTER TABLE SEANCE ADD FOREIGN KEY (salle) REFERENCES SALLE (numero);
```

b)

```
ALTER TABLE FILM ADD COLUMN realisateur VARCHAR(60);
```

c)

```
ALTER TABLE FILM ALTER COLUMN titre TYPE VARCHAR(150);
```

Avec MySQL : ALTER TABLE FILM MODIFY titre VARCHAR(150) NOT NULL; (MODIFY redéclare toute la colonne, il faut donc répéter NOT NULL).

d)

```
ALTER TABLE FILM ALTER COLUMN duree SET NOT NULL;
```

Avec MySQL : ALTER TABLE FILM MODIFY duree INT NOT NULL;

e)

```
ALTER TABLE FILM ADD UNIQUE (titre, annee);
```

f)

```
ALTER TABLE SEANCE ADD CONSTRAINT chk_prix CHECK (prix BETWEEN 0 AND 30);
```

g)

```
ALTER TABLE FILM DROP COLUMN affiche;
```

h)

```
CREATE TABLE EMPLOYE (
   matricule VARCHAR(10) PRIMARY KEY,
   nom       VARCHAR(40) NOT NULL,
   salle     SMALLINT,
   CONSTRAINT fk_employe_salle FOREIGN KEY (salle) REFERENCES SALLE (numero)
);
ALTER TABLE SALLE ADD COLUMN responsable VARCHAR(10);
ALTER TABLE SALLE ADD CONSTRAINT fk_salle_responsable FOREIGN KEY (responsable) REFERENCES
EMPLOYE (matricule);
```

i)

```
ALTER TABLE SEANCE DROP CONSTRAINT chk_prix;
```

Avec MySQL 8 : ALTER TABLE SEANCE DROP CHECK chk_prix;

j) La commande est refusée : les films déjà présents recevraient NULL dans une colonne obligatoire. On ne peut ajouter une colonne obligatoire que si la table est vide ou si la colonne a une valeur par défaut :

```
ALTER TABLE FILM ADD COLUMN langue VARCHAR(2) NOT NULL DEFAULT 'FR';
```

Exercice 8

a) On supprime d'abord les tables qui référencent les autres :

```
DROP TABLE EMPRUNT;
DROP TABLE MEMBRE;
DROP TABLE EXEMPLAIRE;
DROP TABLE ECRIT;
DROP TABLE LIVRE;
DROP TABLE AUTEUR;
```

b) La suppression est refusée : les clés étrangères de ECRIT et de EXEMPLAIRE font référence à LIVRE.

c) Non. ON DELETE CASCADE s'applique quand on supprime des **lignes** de LIVRE, pas quand on supprime la **table**. DROP TABLE LIVRE; reste refusé.

Exercice 9

a) Une table n'a qu'une seule clé primaire, mais peut avoir plusieurs contraintes UNIQUE (identifiants secondaires). Une clé primaire n'accepte jamais NULL, alors qu'une colonne UNIQUE facultative peut en contenir.

b) En SQL, une colonne est facultative par défaut. Sans NOT NULL sur ncli, on pourrait enregistrer une commande qui n'appartient à aucun client.

c) Une salle n'est identifiée que par le couple (étage, numéro) : le numéro seul peut exister à plusieurs étages. La clé étrangère doit reprendre les deux colonnes, prises deux à deux avec le même type.

d) Une clé primaire n'accepte jamais NULL. Or ON DELETE SET NULL doit pouvoir vider membre pour conserver l'historique.

e) La commande est refusée : les données existantes violent l'unicité. Il faut d'abord corriger les doublons.

f) Les deux tables se référencent mutuellement : quelle que soit celle créée en premier, sa clé étrangère viserait une table qui n'existe pas encore. On crée les deux tables, puis on ajoute la seconde clé étrangère avec ALTER TABLE.